
Word-for-word Script: Project 13: Text to Speech

Verbatim 3-person script for the **25-slide** deck (`presentation.html`).
Read your **[NAME]** lines. Hand-offs are marked `→` . These same words are the presenter notes in the deck (press **S** for speaker view), kept in sync.

Speakers: **EMIR** (data / engineering / theory) · **DIMA** (model / training, “I” voice) · **ILYA** (evaluation). Pace ~135 words/min. Pause at `//` . **Target: 15 min presenting** (Q&A is separate, not counted).

Slide 1: Title · EMIR

[EMIR] “Hi, we’re Dima, Emir and Ilya, **Project 13, Text to Speech**. `//` We took a pretrained speech model and **taught it a brand-new custom voice**. Let me start with what we were actually trying to do.”

`→ stays with Emir`

Slide 2: The goal · EMIR

[EMIR] “The pretrained model already produces clean speech, so ‘make good speech’ isn’t the goal, it already does that. `//` The goal is to **change the voice**: take the model and fine-tune it to speak as a **B1 battle droid**. `//` We picked a droid on purpose, a subtle voice makes a demo ambiguous, but a droid is obvious, you can **hear** the change instantly.”

`→ HANDS TO DIMA`

Slide 3: Where to start: the ideas · DIMA

[DIMA] “Thanks. Running a pretrained model isn’t really a project, so I wanted to **change the voice itself**. I brainstormed four directions: // a **B1 droid voice**, a timbre change you hear instantly; a **heavy accent**, which is phoneme-level; **emotions**, which is prosody; and **stuttering**, which is rhythm. // I dropped stuttering and emotions as too shallow, kept the **droid** as the core, and left the accent as future work.”

→ stays with Dima

Slide 4: Three ways to change a TTS voice · DIMA

[DIMA] “There are three ways to change the voice. // I can **pre-process** the input, useful for an accent, but limited. I can **post-process** the output audio, I tried that and it sounds robotic and fake. // Or I **bake the voice into the model itself** by fine-tuning on data in that voice, that’s the real, proper way, and that’s what I did. The droid becomes the model’s own voice.”

→ stays with Dima

Slide 5: Building the dataset · DIMA

[DIMA] “To fine-tune I need lots of paired text and audio in the droid voice, you can’t record 24 hours of a droid. So **I built the data**: // I took all of LJSpeech and ran it through **RVC**, a voice-conversion model, which changes the timbre to the B1 droid but keeps the words intact, so the original transcripts still match. // That gave me **13,100 paired clips** instantly, and I **published** the dataset on the Hugging Face Hub. Emir will take the model from here.”

→ HANDS TO EMIR

Slide 6: Why VITS · EMIR

[EMIR] “Thanks Dima. We chose **VITS** for three reasons. // It’s **end-to-end**, text straight to waveform in one model, no separate vocoder to train and keep in sync. // It gives us **phoneme-level access**, which we wanted for a

Russian-accent idea later. // And there's a **strong pretrained checkpoint** to fine-tune from. Under the hood it's a conditional VAE with a normalizing flow, a GAN decoder, and a stochastic duration predictor."

→ stays with Emir

Slide 7: The generator: why it isn't enough · EMIR

[EMIR] "Here's the generator. During training it gets **two inputs**, the text and the real audio, and learns to rebuild the waveform, comparing on the mel-spectrogram with an L1 loss. // The problem: train on reconstruction **alone** and the audio comes out blurry and muffled, L1 just can't capture fine detail. That's exactly the gap a **discriminator** fills. // One thing to stress: the real audio is only used in **training**, to teach the text side; at inference there's no audio, it's purely text to speech."

→ stays with Emir

Slide 8: Discriminator: built from scratch · EMIR

[EMIR] "Hugging Face only ships VITS for **inference**, there's no discriminator in the package, so we wrote one **from scratch**, HiFi-GAN style. // There are two: a **multi-period** discriminator that folds the waveform by prime periods to catch pitch and harmonics, and a **multi-scale** discriminator that looks at three time scales. // Both also expose their internal feature maps, which gives us a **feature-matching** loss. Without this whole piece the decoder just never learns to produce realistic audio."

→ stays with Emir

Slide 9: Lining up text & audio: MAS · EMIR

[EMIR] "One piece worth explaining, **alignment**. We have maybe ten text tokens but hundreds of audio frames, and nothing tells us which frames belong to which token. // VITS solves this with **Monotonic Alignment Search**, MAS. It's a dynamic-programming search, like Viterbi, that finds the best **monotonic** mapping from text to frames, monotonic meaning speech moves left to right and can't jump back or skip, so it can't babble. // As a

bonus, the alignment tells us how many frames each token should last, which is exactly the target for the **duration loss**. With that, Dima will take you through how he trained it.”

→ HANDS TO DIMA

Slide 10: How training works · DIMA

[DIMA] “One training step: encode text to a prior, encode the real audio to a latent, run the flow, use Monotonic Alignment Search to line up text with audio frames, slice a short segment, decode it to a waveform, and compute the losses. // Then it’s a standard **GAN alternation**, the discriminator trains on real versus detached fake, then the generator trains on **five losses** to fool it, with reconstruction weighted 45×. // I kept the messy GAN plumbing, two optimizers, detach, clipping, mixed precision, accumulation, inside **decorators**, so the actual step logic reads cleanly and the wiring lives separately.”

→ stays with Dima

Slide 11: Abstraction: keeping the loop readable · DIMA

[DIMA] “One engineering thing I am proud of. The GAN loop is messy, two optimizers, detach, zero-grad, backward, clipping, the AMP scaler, accumulation, checkpointing, all tangled around two tiny core steps. // So I moved all that wiring into **decorators** and left only the actual discriminator-step and generator-step logic in the methods. // Read the method and the algorithm is linear; need the plumbing, read the decorator. Same idea as `torch.nn.Module` hiding backward, the complexity does not disappear, it goes where it belongs.”

→ stays with Dima

Slide 12: Where & how I trained · DIMA

[DIMA] “On compute, we started on **Colab** and it fought us: sessions dropped, the free tier caps around 15 minutes, disk filled with checkpoints. // So I moved to my **local RTX 5060**, 8 gigs. To fit 8 gigs I used batch size 1 with **gradient accumulation** for a larger effective batch, mixed-precision

fp16, and zero dataloader workers because the Windows loader kept crashing. // The final clean run was **5 epochs, about 9.7 GPU-hours**, roughly 1.9 hours per epoch, and that's on top of maybe three times that in failed runs and debugging."

→ stays with Dima

Slide 13: Training cost & config · DIMA

[DIMA] "The numbers. The final clean run was 5 epochs, about 1.9 hours each, roughly **9.7 GPU-hours** total, plus maybe three times that in failed runs and debugging, all on my local RTX 5060. // Key config: learning rate **1e-4**, much gentler than the from-scratch 2e-4; batch size 1 with gradient accumulation; a decoder crop of 8192; gradient clip at 10; a discriminator warm-up of 1500 steps; mixed precision; and reconstruction weighted 45."

→ stays with Dima

Slide 14: Problems & fixes · DIMA

[DIMA] "This was the hardest and most interesting part, at first **nothing converged**. The reconstruction loss was stuck, the duration loss was in the hundreds, and the voice came out way too fast. Three bugs. // **One:** the duration loss was about a hundred times too large because it was averaged wrong, fixing the normalization dropped it from three hundred to under two. // **Two:** gradient clipping was so tight it killed the generator's updates, loosening it let reconstruction finally drop. // **Three:** a fresh, untrained discriminator was corrupting the pretrained decoder, so I warmed up the discriminator alone for 1500 steps and lowered the learning rate. // After that, real training: reconstruction fell from 0.85 to 0.40 and the rhythm became correct."

→ stays with Dima

Slide 15: Training curves: the voice is learned · DIMA

[DIMA] "Here's what my training looked like once it worked. On the **left**, reconstruction drops sharply then plateaus around 0.42, the core voice is learned early on. // On the **right**, the duration loss settles at a healthy 1.6 after

I fixed the normalization bug, which gives correct rhythm. // So the voice side is healthy. Next: did the **GAN**, and the discriminator I built from scratch, train stably?”

→ stays with Dima

Slide 16: Training curves: the GAN stayed balanced · DIMA

[DIMA] “And it did. These are the two GAN losses. On the **left** the generator’s adversarial loss oscillates around 4, it never runs away. On the **right** the discriminator sits stable around 5.5; the warm-up kept it from overpowering the pretrained decoder. // Neither one collapses, so the adversarial training reached a healthy equilibrium, the discriminator I wrote from scratch is doing exactly its job. // But a stable loss curve does **not** prove the speech is actually intelligible, that’s what evaluation checks. Over to Ilya.”

→ HANDS TO ILYA

Slide 17: Evaluation: what we measure · ILYA

[ILYA] “We evaluate on a **held-out** set, the last 5% of the data the model never trained on. Three metrics, all **lower-is-better**. // **WER and CER** are intelligibility: we run Whisper, an independent speech recognizer, on our audio and check how well it transcribes back to the text. // **MCD** is spectral, how close our output is to the actual B1 droid reference, i.e. did it learn the timbre. // And **F0 RMSE** is prosody, the pitch difference. The assignment asks for error rates plus at least one spectral metric; we report two. Before and after is just a checkpoint swap on the same prompts.”

→ stays with Ilya

Slide 18: Why these metrics, and the B1 twist · ILYA

[ILYA] “Why these three. **WER and CER** from Whisper measure intelligibility, and they work for any voice including the droid. **MCD** measures spectral closeness to a target voice, and it is the key metric here, it tells us whether we learned the droid timbre. **F0 RMSE** is pitch. // And here is the **B1 twist** that matters: we compute MCD and F0 against the **droid reference** from our dataset, not against the original female LJSpeech, because comparing a

droid to a female voice would just measure how un-droid-like we are. For a deliberate voice change, the spectral metric only makes sense against the **new target.**”

→ stays with Ilya

Slide 19: How we evaluate · ILYA

[ILYA] “How we actually evaluate. We hold out the last 5% of the data, which the model never trains on. // Before and after is a single **checkpoint swap** on the same prompts, so it is apples-to-apples. // I built the evaluation as a small **contract-first module**: feed it a manifest of texts and reference audio, and it returns one result object with WER, CER, MCD and F0 in a single pass. // And the ASR is **Whisper**, frozen and off-the-shelf, so it never touches training and there is no circularity.”

→ stays with Ilya

Slide 20: Results: before / after · ILYA

[ILYA] “These are the real numbers, on 20 held-out clips. The headline is the **trade-off**. // **MCD**, the distance to the droid reference, drops from **10.5 to 6.2 decibels**. That’s hard proof the fine-tuning worked: the model’s output is now spectrally much closer to the droid voice than the pretrained model was. // The cost shows up in intelligibility: **word error rate rises from 9 to 29 percent**, it’s understandable but less clean than the base. Pitch is about the same. // So we measurably **gained the new voice, at a measurable cost to clarity**, and reducing that cost is our main future work.”

→ stays with Ilya

Slide 21: Demo · ILYA

[ILYA] “Let’s hear it, the **same sentence**, before and after, so you hear the voice change directly. // First the **base** VITS voice. // *(play 1)* // And now our **fine-tuned** model saying the exact same line, in the droid voice. // *(play 2)* // Same words, the voice changed, and it stays understandable. On some short prompts it still degrades, which is the next slide.”

Pre-load both players before the talk. Test room audio. Backup: clips are in `assets/audio/` if the players don't fire on the projector.

→ stays with Ilya

Slide 22: Failure modes & limitations · ILYA

[ILYA] “Honest limitations. // First, the **intelligibility cost** we just saw, and on some short punchy prompts the durations collapse and it comes out garbled, here's one: ‘Roger roger, all units...’ should be five seconds, but it comes out as under a second of mush. // *(play the collapse clip)* // Second, it's not a hard sci-fi robot, **RVC transfers a speaker's timbre**, not the ring-modulator buzz, so it's a brighter voice rather than mechanical; that buzz would be a separate effect. // Third, typical **GAN artifacts** early on, and quality drops if you train past the best epoch. And the **Russian accent** was going to be input pre-processing, left as future work. Emir, on the engineering.”

→ HANDS TO EMIR

Slide 23: Engineering: a team of 3 · EMIR

[EMIR] “A word on how three of us built this without colliding. // **Contract-first** design: the data, the model, and the evaluation only talk through small fixed interfaces, a manifest and an evaluation-result object, never each other's internals. // That let us work **in parallel** and kept the model **swappable**. // We also have an offline mock and a smoke test, so we can check the whole pipeline wires together in seconds without a GPU, plus pinned dependencies, a fixed seed, and lint for reproducibility.”

→ stays with Emir

Slide 24: Takeaways & future work · EMIR

[EMIR] “To wrap up. // We **built and published** our own droid dataset with RVC, and **fine-tuned VITS** onto that voice. // We **re-implemented the whole GAN training** stack from scratch, discriminator, losses, the loop, because the library ships inference-only. // The result is a clear, measured **trade-off**: the

timbre moved to the droid by MCD, at a cost in intelligibility by WER. // And the clear next step to cut that WER is to **freeze the text and timing front-end** during fine-tuning, which keeps the durations stable, plus finally adding the Russian accent. Thanks, questions?”

→ ALL, open the floor

Slide 25: Thank you · Q&A · ALL

Route by ownership: **model** → Dima, **evaluation** → Ilya, **data & engineering** → Emir. Repeat each question before answering.

Timing summary (25 slides, 15 min target)

Speaker	Slides	Count	Approx
Emir	1, 2, 6, 7, 8, 9, 23, 24	8	~5 min
Dima	3, 4, 5, 10, 11, 12, 13, 14, 15, 16	10	~6 min
Ilya	17, 18, 19, 20, 21, 22	6	~4 min
All	25 (Q&A)	1	separate

Total ≈ 15 min presenting + Q&A separate. 25 slides in 15 min is slide-heavy (~36 s/slide), so keep moving: Dima talks fast and lets the visuals carry; Emir and Ilya carry the substance.

If running long (cut first)

- Merge the two curve slides (15+16) into one mention.
- Fold “Training cost & config” (13) into “Where & how I trained” (12) as one line.
- Trim “Three ways” (4) to one line; drop the MAS detail (9) to a one-liner and lean on Q&A.
- Collapse “Why these metrics” (18) into “Evaluation” (17), it restates the same three metrics.